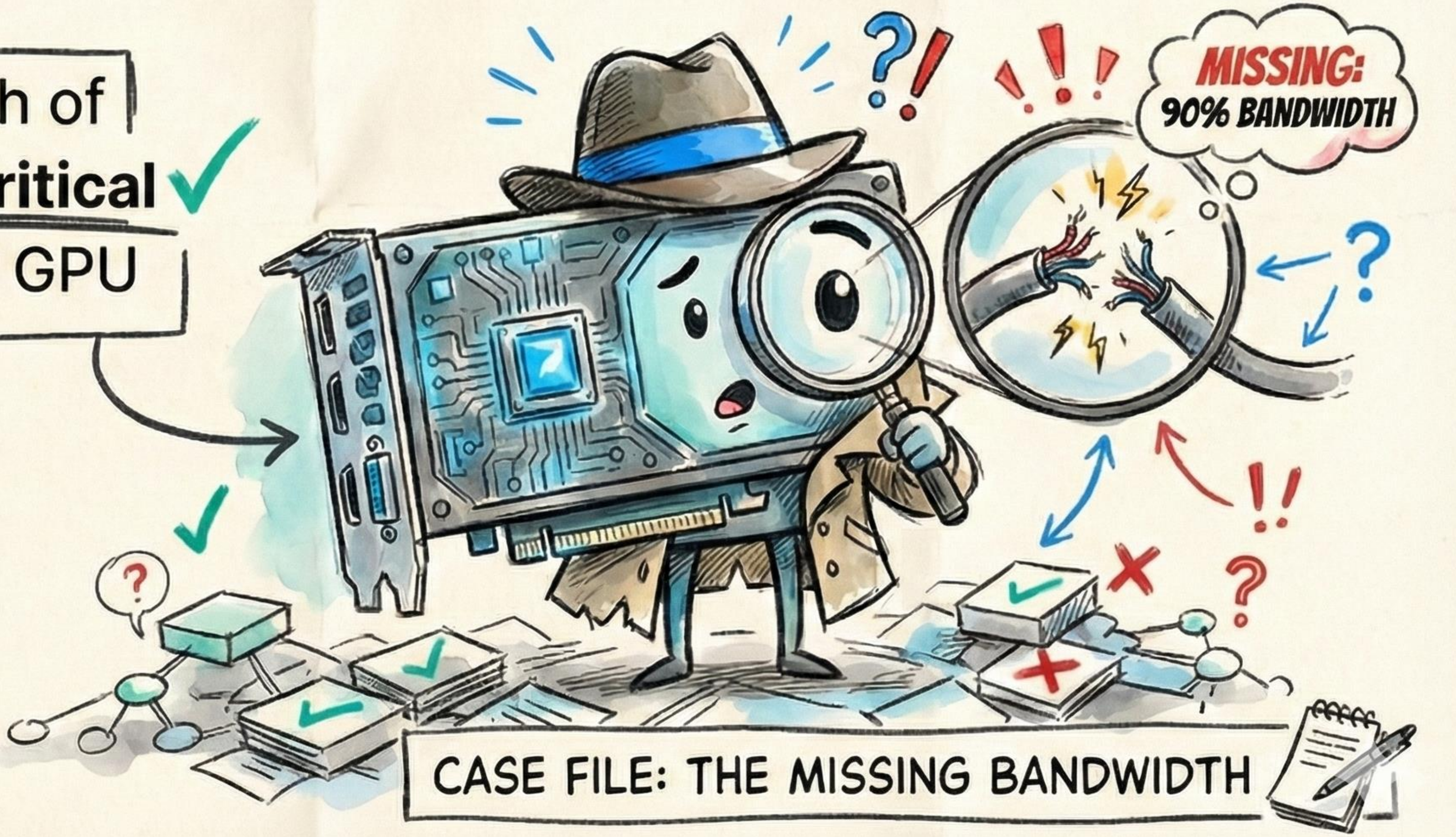
THE GREAT PERFORMANCE HEIST: DEBUGGING METAX MCCL

Why did our multi-node GPU cluster lose 90% of its speed?

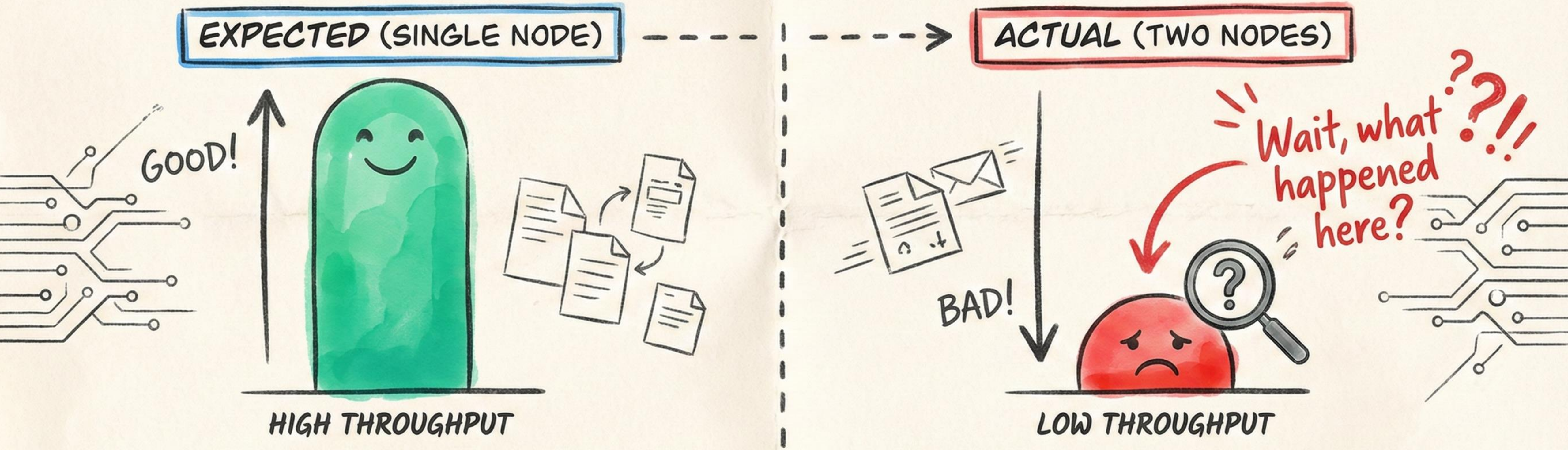
A systematic walkthrough of identifying and **fixing a critical bottleneck** in distributed GPU communication.



George Zheng Wang, SAA,
zhengwan@redhat.com

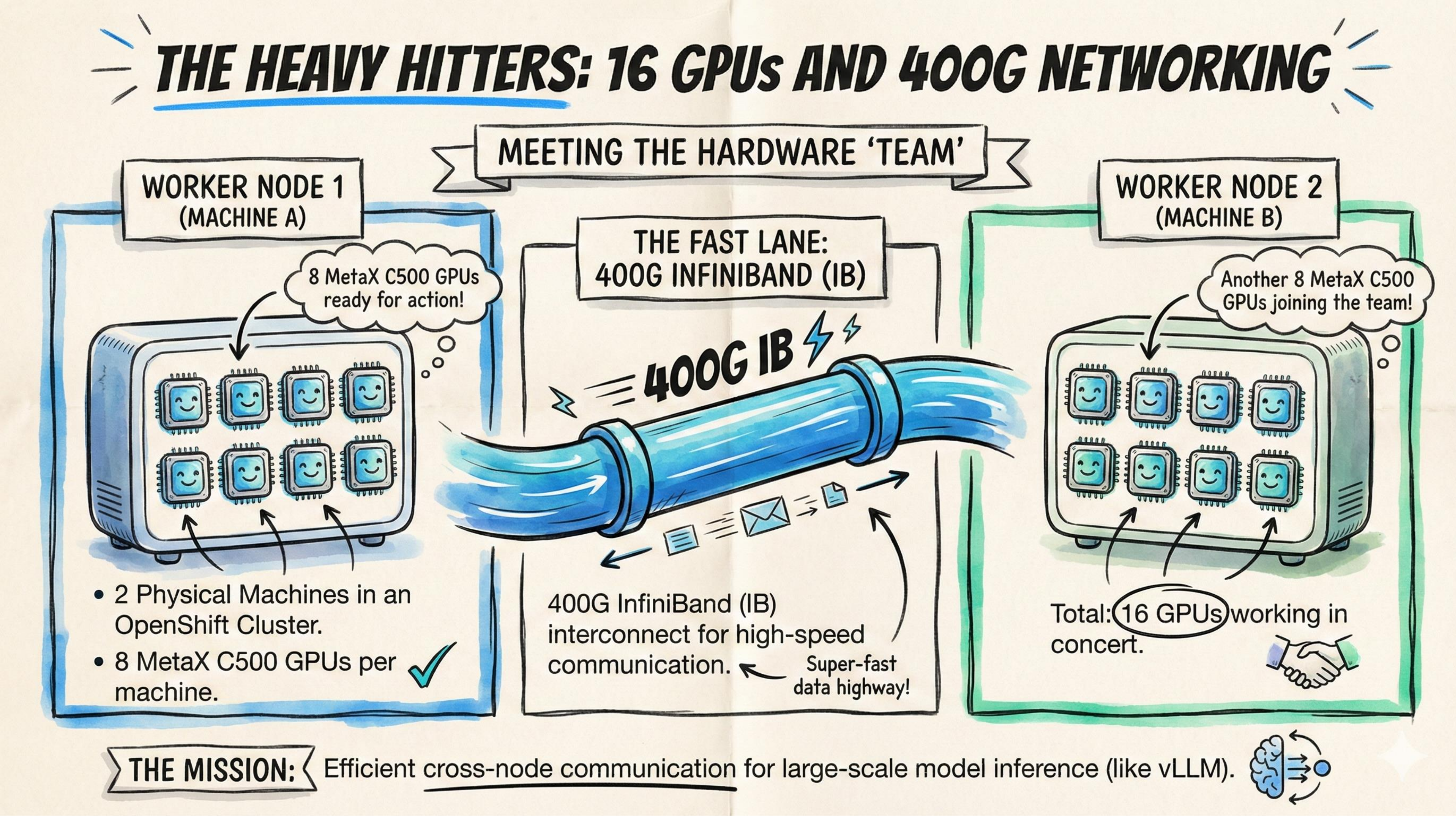
THE 10x PERFORMANCE DROP

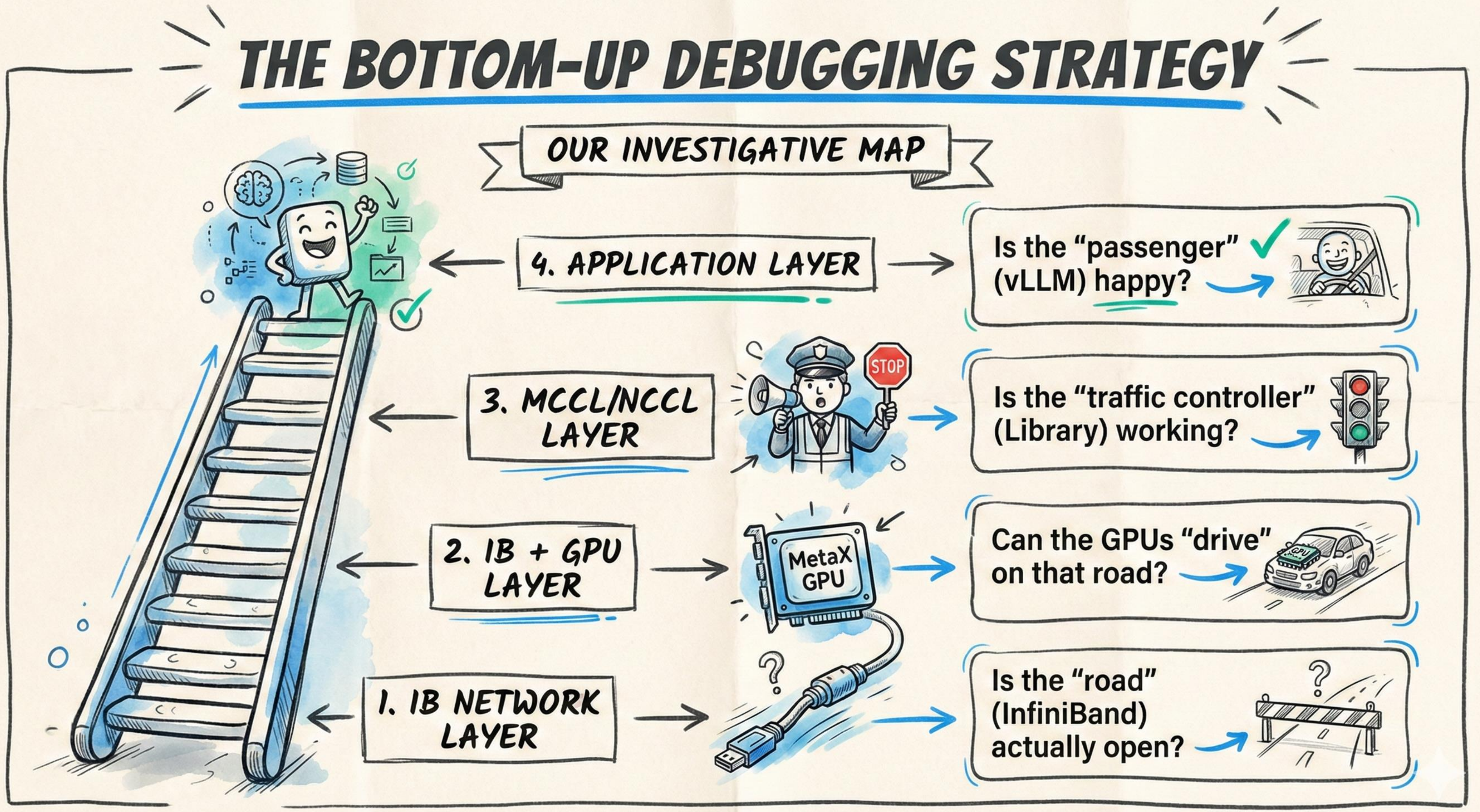
EVERYTHING WAS FINE... UNTIL WE ADDED A SECOND NODE.



- **Single-node inference:** Performs as expected.

- **Two-node configuration:** Throughput plummeted nearly tenfold.
- Average bus bandwidth dropped to a measly 4.57 GB/s.


THE HEAVY HITTERS: 16 GPUs AND 400G NETWORKING
MEETING THE HARDWARE 'TEAM'
WORKER NODE 1 (MACHINE A)
8 MetaX C500 GPUs ready for action!
• 2 Physical Machines in an OpenShift Cluster.
• 8 MetaX C500 GPUs per machine.
THE FAST LANE: 400G INFINIBAND (IB)
400G IB
400G InfiniBand (IB) interconnect for high-speed communication.
Super-fast data highway!
WORKER NODE 2 (MACHINE B)
Another 8 MetaX C500 GPUs joining the team!
Total: 16 GPUs working in concert.
THE MISSION: Efficient cross-node communication for large-scale model inference (like vLLM).

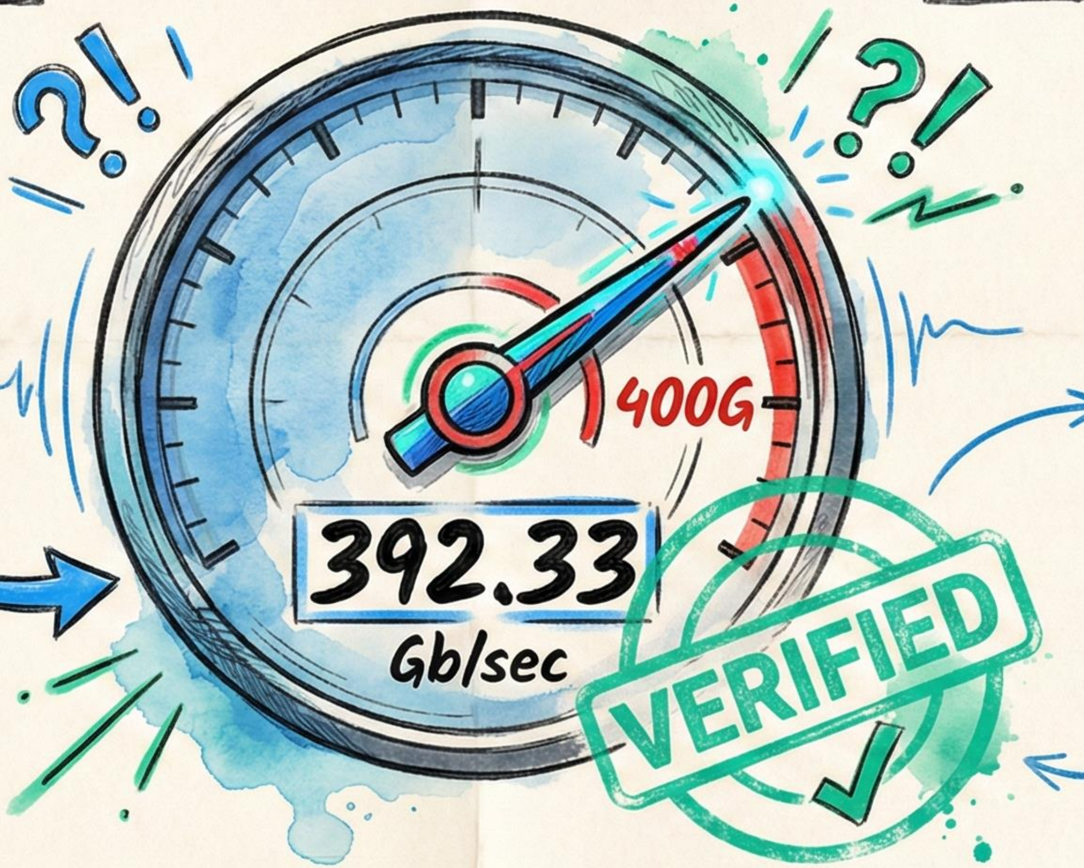

THE BOTTOM-UP DEBUGGING STRATEGY
OUR INVESTIGATIVE MAP
4. APPLICATION LAYER
Is the "passenger" (vLLM) happy?
STOP
3. MCCL/NCCL LAYER
Is the "traffic controller" (Library) working?
2. IB + GPU LAYER
MetaX GPU
Can the GPUs "drive" on that road?
1. IB NETWORK LAYER
Is the "road" (InfiniBand) actually open?

TESTING THE RAW IB BANDWIDTH

LAYER 1 — THE 'ROAD' IS PERFECT

THE TOOL: ib_write_bw (Testing node-to-node without GPUs).

THE RESULT: Peak performance hit 392.33 Gb/sec.



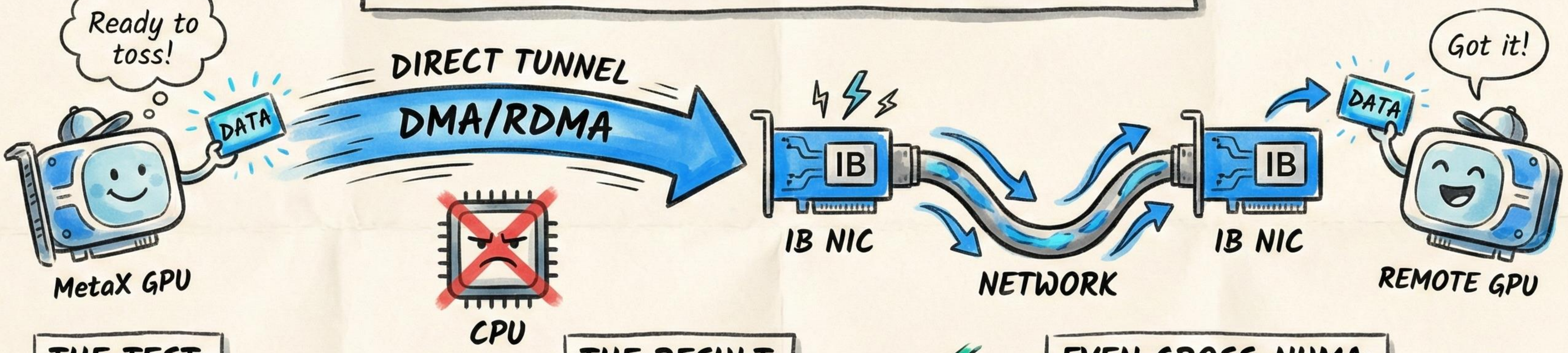
MESSAGE SIZE	SPEED (Gb/sec)
64 B	1.2 Gb/s
512 B	15.6 Gb/s
4 KB	89.4 Gb/s
64 KB	210.8 Gb/s
1 MB	380.1 Gb/s
16 MB	392.33 Gb/s

PEAK PERFORMANCE!

CONCLUSION: The physical network layer is functioning correctly and reaching near line rate.

TESTING GPU-TO-NIC DIRECT ACCESS

LAYER 2 — THE GPUs CAN 'DRIVE' JUST FINE



THE TEST:

Moving data from local GPU memory to a remote GPU's memory via DMA.

THE RESULT:

~392 Gb/sec

Again, we hit near line-rate performance. WOW!

EVEN CROSS-NUMA:

Communication between a GPU GPU on one CPU and a NIC on another CPU was still fast.

CPU ⟷ CPU NO PROBLEM!

CONCLUSION: The GPUs can efficiently talk to the Network Cards directly, even across complex system architectures.

THE BOTTLENECK IS IN THE McCL LAYER

LAYER 3 — THE “TRAFFIC CONTROLLER” IS FAILING

THE EVIDENCE:

	SPEED TEST
✓	Raw IB Network Tests: **400 Gb/s** Fast!
✗	MCCL Multi-Node Test: **4.57 GB/s** Slow!

Even with a single GPU per node, performance only slightly improved to 6.97 GB/s.

THE MYSTERY:

If the hardware is fast, why is the library slow?

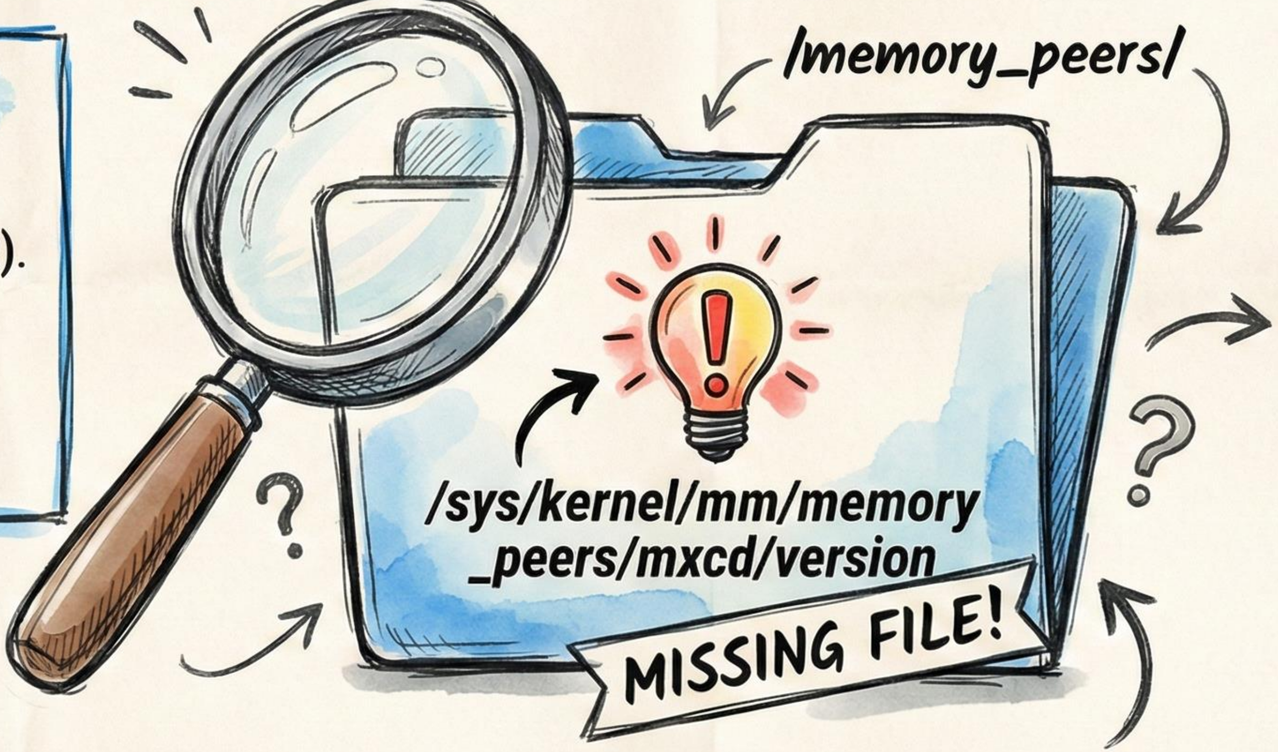
CONCLUSION: The MCCL communication library is the source of the significant performance degradation.

A MISSING FILE AND A BROKEN LINK

THE BREAKTHROUGH — THE CASE OF THE MISSING FILE

WHY IT MATTERS:

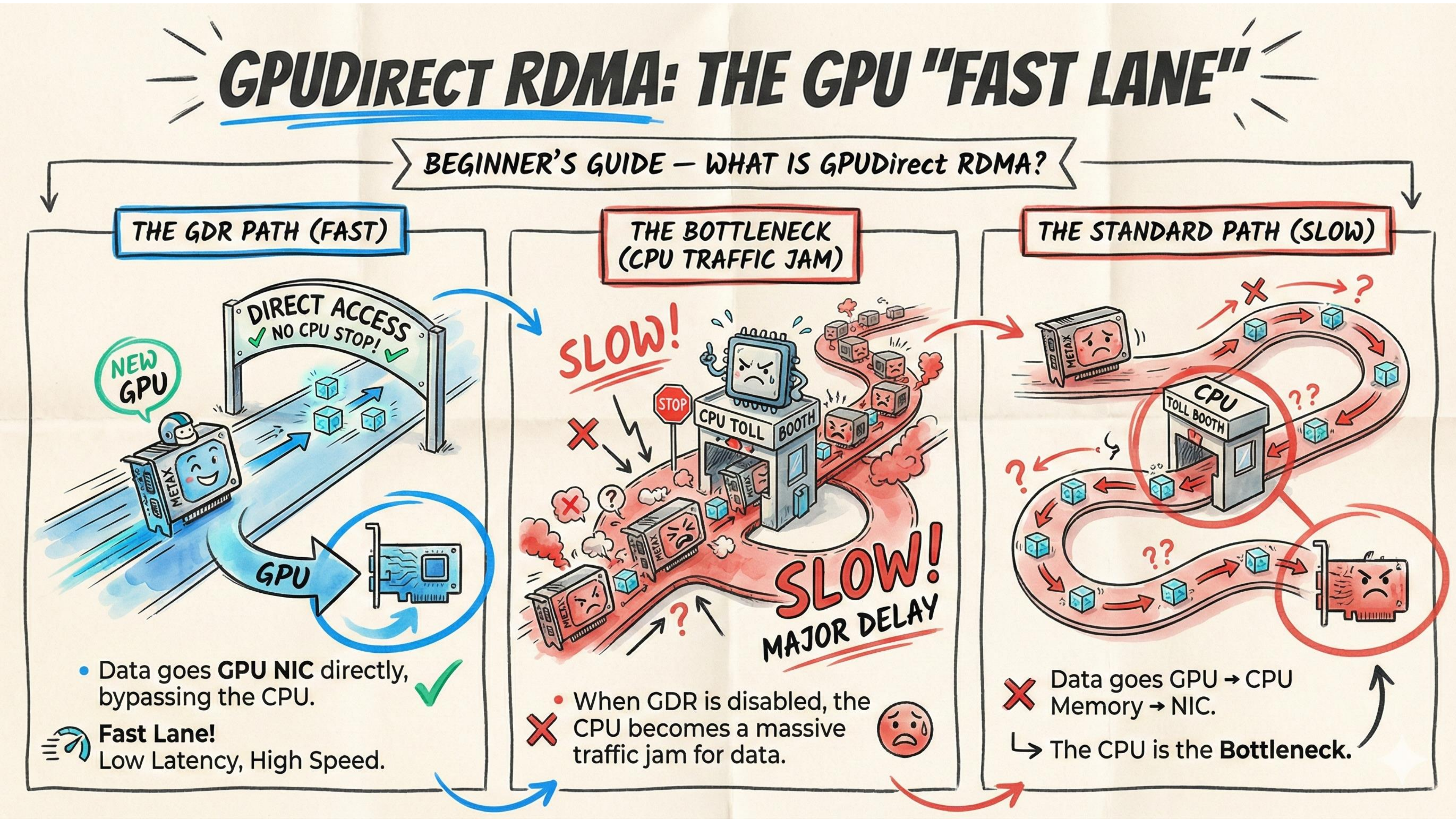
This file is required for **GPUDirect RDMA (GDR)**.



THE CONSEQUENCE:

Without this file, the MCCL library couldn't detect GDR and 'forgot' how to talk to the GPUs directly.

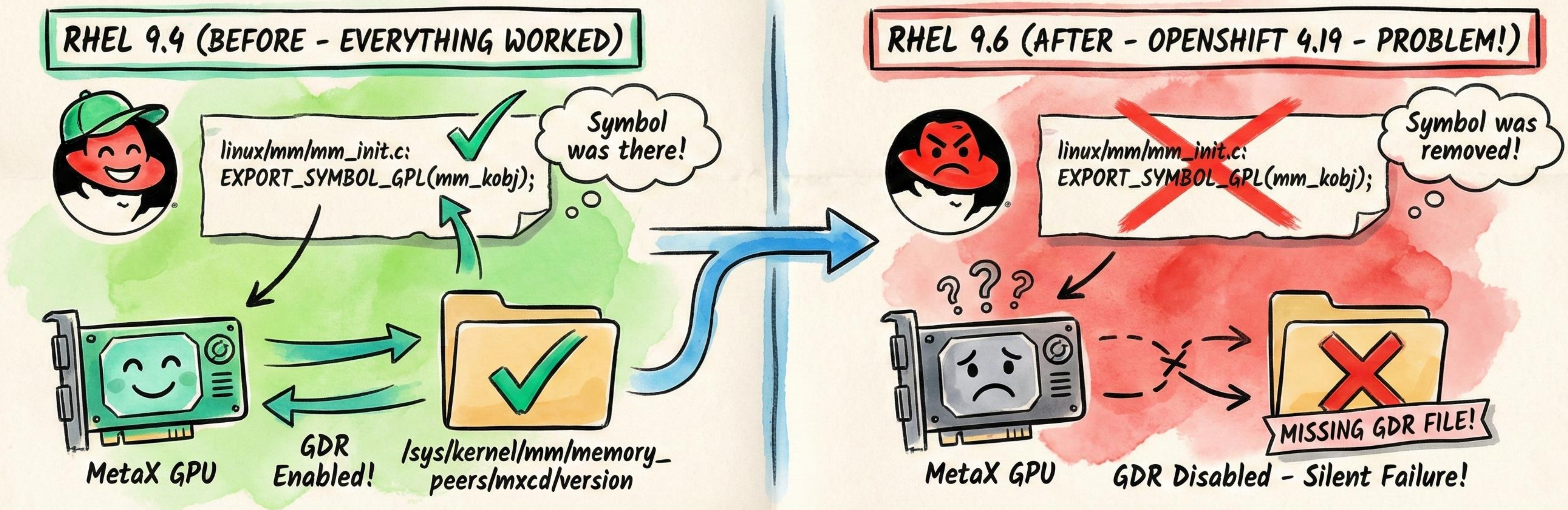
CONCLUSION: The missing GDR kernel file caused the MCCL library to fail in direct GPU communication, leading to the performance crash.



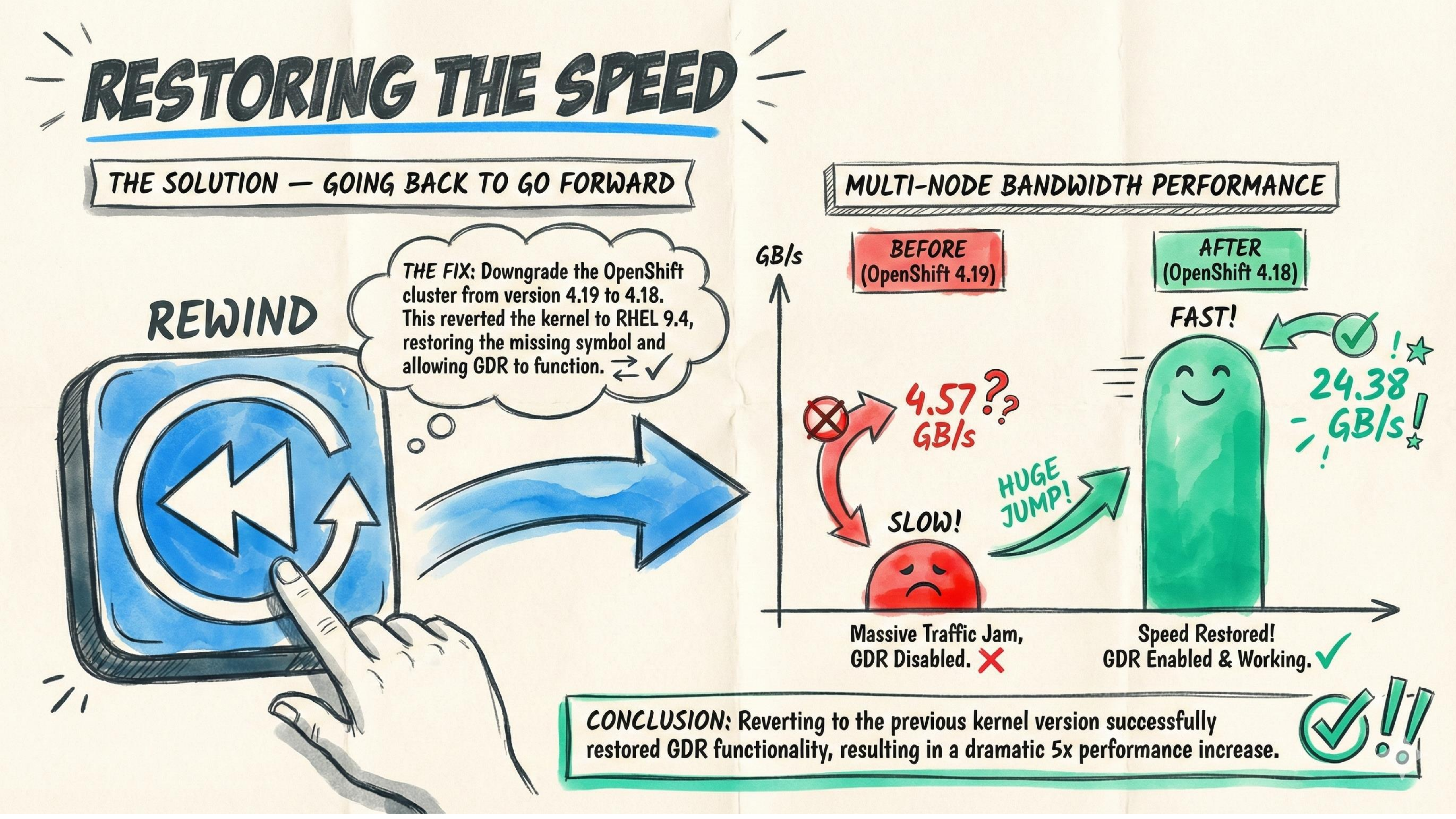
GPUDirect RDMA: THE GPU "FAST LANE"
BEGINNER'S GUIDE — WHAT IS GPUDirect RDMA?
THE GDR PATH (FAST)
DIRECT ACCESS
NO CPU STOP!
NEW GPU
METAX
GPU
Data goes GPU NIC directly, bypassing the CPU.
Fast Lane!
Low Latency, High Speed.
THE BOTTLENECK (CPU TRAFFIC JAM)
SLOW!
STOP
CPU TOLL BOOTH
METAX
SLOW!
MAJOR DELAY
When GDR is disabled, the CPU becomes a massive traffic jam for data.
THE STANDARD PATH (SLOW)
METAX
CPU
TOLL BOOTH
Data goes GPU → CPU Memory → NIC.
The CPU is the Bottleneck.

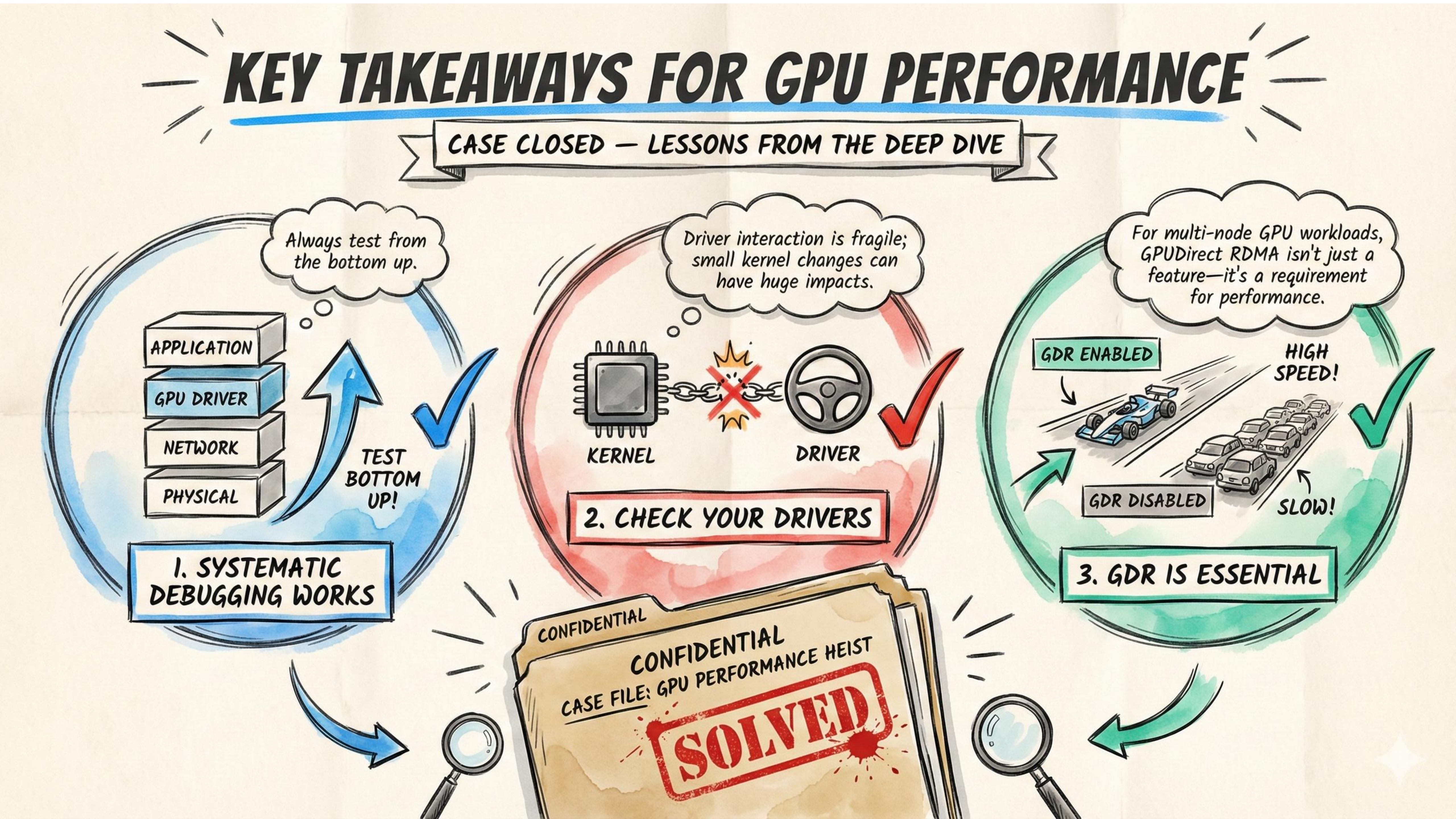
THE RHEL 9.6 SURPRISE

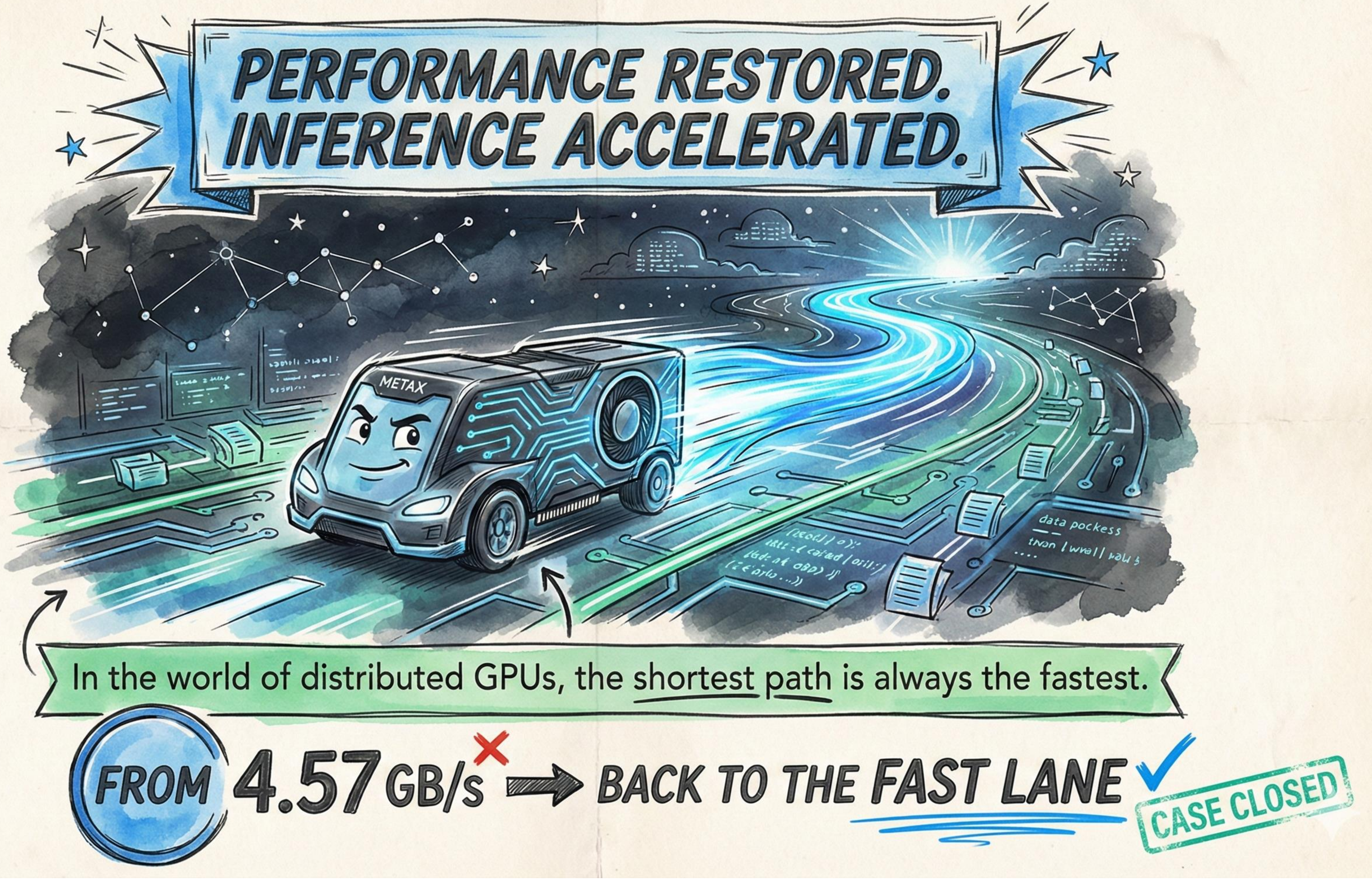
THE ROOT CAUSE — A SURPRISE KERNEL PATCH



CONCLUSION: The kernel patch removing the `mm_kobj` symbol caused a silent failure that crippled GPU performance.


RESTORING THE SPEED
THE SOLUTION — GOING BACK TO GO FORWARD
REWIND
THE FIX: Downgrade the OpenShift cluster from version 4.19 to 4.18. This reverted the kernel to RHEL 9.4, restoring the missing symbol and allowing GDR to function.
MULTI-NODE BANDWIDTH PERFORMANCE
GB/s
BEFORE (OpenShift 4.19)
AFTER (OpenShift 4.18)
FAST!
4.57 GB/s
24.38 GB/s
HUGE JUMP!
SLOW!
Massive Traffic Jam, GDR Disabled.
Speed Restored! GDR Enabled & Working.
CONCLUSION: Reverting to the previous kernel version successfully restored GDR functionality, resulting in a dramatic 5x performance increase.


KEY TAKEAWAYS FOR GPU PERFORMANCE
CASE CLOSED — LESSONS FROM THE DEEP DIVE
Always test from the bottom up.
APPLICATION
GPU DRIVER
NETWORK
PHYSICAL
TEST BOTTOM UP!
1. SYSTEMATIC DEBUGGING WORKS
Driver interaction is fragile; small kernel changes can have huge impacts.
KERNEL
DRIVER
2. CHECK YOUR DRIVERS
For multi-node GPU workloads, GPUDirect RDMA isn't just a feature—it's a requirement for performance.
GDR ENABLED
HIGH SPEED!
GDR DISABLED
SLOW!
3. GDR IS ESSENTIAL
CONFIDENTIAL
CONFIDENTIAL
CASE FILE: GPU PERFORMANCE HEIST
SOLVED


PERFORMANCE RESTORED.
INFERENCE ACCELERATED.
METAX
In the world of distributed GPUs, the shortest path is always the fastest.
FROM 4.57 GB/s → BACK TO THE FAST LANE
CASE CLOSED